

9

Securing ASP.NET Core

Correctly securing your APIs is essential. In this chapter, we will learn how to secure an ASP.NET Core 5 Web API application because it is also the responsibility of the developer, who is building the backend application to protect the APIs. There are two available frameworks in .NET that you can use to secure your application, and there are also cloud identity providers.

We will tackle the following topics:

- Understanding ASP.NET Core Identity
- Introducing IdentityServer4
- Getting to know **customer identity and access management (CIAM)**
- Authenticating using JWT

Technical requirements

Here is what you need to complete this chapter:

- Visual Studio 2019, Visual Studio for Mac, or Rider
- Postman
- JWT Debugger: <https://jwt.io>

Here is the link to the finished repo of this chapter: <https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter09>.

Understanding ASP.NET Core Identity

ASP.NET Core Identity is an open source identity framework in .NET. It provides the .NET application with a way to implement authentication, to determine somebody's identity and authorization, which details the limits to the user's access. It also allows a .NET application to manage user accounts. Using Identity in an application is recommended so your application complies with security best practices and precautions. ASP.NET Core also saves development time, so you can just focus on your business logic.

ASP.NET Core Identity features

The major features that ASP.NET Core Identity provides are as follows:

- User accounts and password management.
- **Multi-factor authentication (MFA)** and **two-factor authentication (2FA)**.
- Login and logout features.
- External authentication providers, such as Facebook, Google, Twitter, and Microsoft account.
- Customize the Identity framework based on your needs.

Developers need to understand how authentication flows in ASP.NET Core Identity and how it works. ASP.NET Core Identity uses a claims-based security scheme that was introduced back in .NET version 4.5.

A **claim** is an identity's property, and it includes a key-value pair. An identity can have one or more claims associated with it. For example, a student user in a school application can have multiple claims defining who they are. These claims can include names, schoolEmails, or dateOfBirths. Every claim must have a value that's unique to that student.

The `ClaimsIdentity` class represents an identity and all of its claims. A user in the ASP.NET Core application can have more than one identity through the help of the `ClaimsPrincipal` class. `ClaimsPrincipal` comprises one or more `ClaimsIdentity` classes; hence, it can have one or more claims.

To give a better overview of `ClaimsIdentity` and `ClaimsPrincipal`, let me show you a diagram, *Figure 9.1*, as follows:

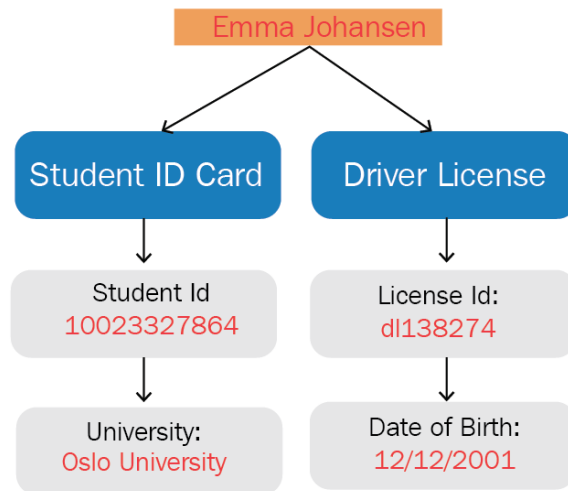


Figure 9.1 – Two forms of identification

To illustrate, see the preceding *Figure 9.1*, where a student may have two forms of identification: their student ID card and their driver's license card. The student ID number and name of the university or school are visible on the student ID card. Their driver's license card has the license ID and their date of birth. Each identity is flexible enough to allow access to different resources. `ClaimsPrincipal` inherits the list of claims from the associated identities. In this scenario, the driver's license claims and student ID card claims identify the student user.

Cookie authentication is the most common authentication in ASP.NET Core applications. It's good to know that ASP.NET Core has built-in middleware that enables the application to utilize cookie-based authentication.

Also, to give a better overview of cookie authentication, let me show you a diagram, *Figure 9.2*, as follows:

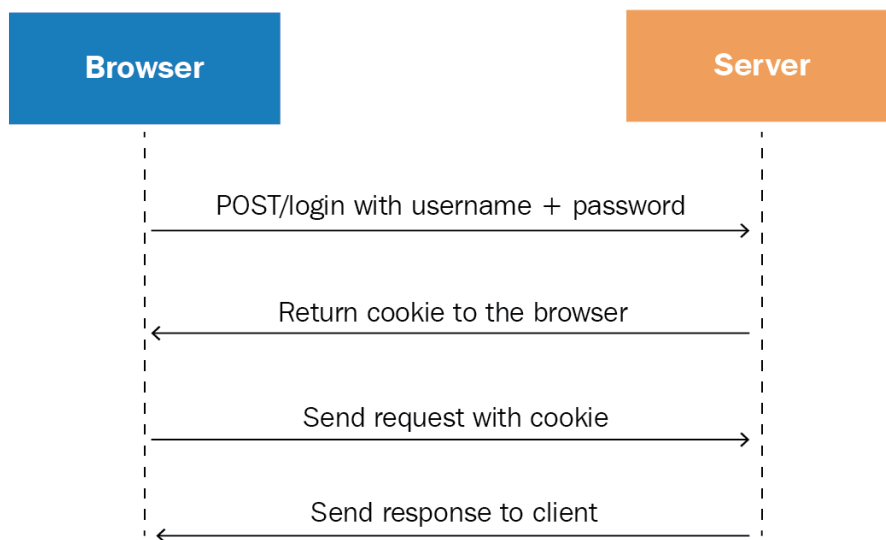


Figure 9.2 – ASP.NET Core cookie middleware

The preceding diagram is an overview of how a simple ASP.NET Core cookie middleware works. The following steps are what happens under the hood:

1. `ClaimsPrincipal` and all its inherited claims are serialized into a cookie after a successful login of the user.
2. The browser saves an encrypted cookie from the server.
3. The cookie is sent via the request header and validated by the server on every next request.
4. The cookie is then used to recreate `ClaimsPrincipal` after successful verification of it. The principal then gets attached to the HTTP context user property.
5. Finally, all the user information and claims linked to the principal can be accessed by the application code.

That's a quick introduction to ASP.NET Core Identity; now, let's go on to `IdentityServer4`.

Introducing IdentityServer4

Security Token Service (STS) is a critical component of token-based authentication. Other terms that you might encounter are identity provider or authorization server. It's a piece of software that generates and issues security tokens to client-side applications. Client-side applications redirect user verification to STS to handle it. The token is encrypted and signed to ensure that the token is protected from any tampering.

The encryption in the application uses a private key kept by the token service, while the public key for decrypting the token is shared with client-side applications. In this way, the client applications trust that the token came from the right token service. Standards for providing identity information, such as OpenID Connect, are used by token services.

When it comes to consuming token services, there are different options. You can use a pre-built service such as **Active Directory Federation Services (ADFS)** to build your custom service using the IdentityServer framework.

IdentityServer4 is an open source framework for token authentication. At the time of writing, IdentityServer4 is the latest recommended version for ASP.NET Core 5 applications:

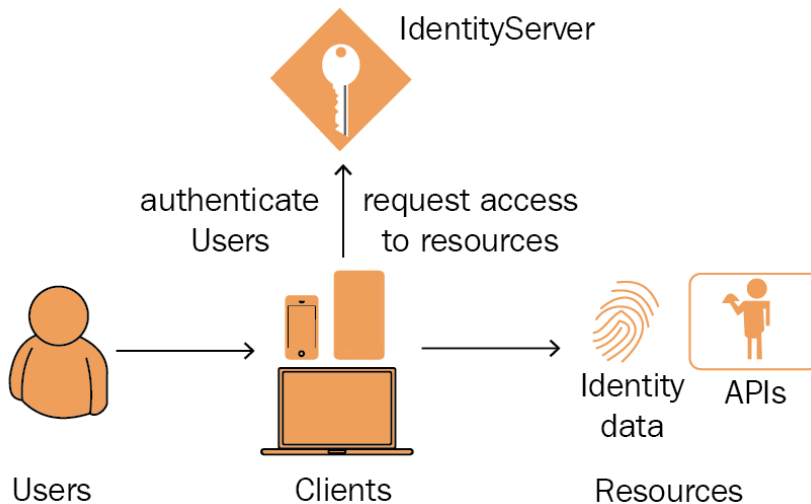


Figure 9.3 – Basic structure of a security system

The preceding diagram explains the fundamental structures of a security system. A client is an application that requests tokens from IdentityServer4. It can be a mobile app or web app, for example. IdentityServer4 has to know the client applications that are permitted to use it by registering or including them in a collection of entities or stores. This collection can be persisted or saved in a database or memory.

Next are the end users who use or interact with client-side applications. The users also have to be registered through IdentityServer. A unique identifier, along with username, and password credentials, are samples of identity data that define users. Users can also have one or multiple associated claims. The identity resources and the API resources are the stuff that needs protection. Identity resources are details about identity, such as user claims. API resources, on the other hand, represent protected functionality, such as Web APIs. Resources also have to be registered; they need to be registered in an identity server store. Two types of tokens are generated along with the two resource types.

Information is sent with the identity token when IdentityServer4 authenticates the user. The client sends forward the access token to the API, which allows or grants access to protected functionality or data when access is requested for the API resource, and the access token is successfully issued.

IdentityServer4 is a middleware; it uses industry-standard **JSON Web Tokens (JWTs)**, and it implements two standard protocols, OAuth 2 and OpenID Connect. **OAuth 2** is an open standard for authorization; it makes sure that a user has the right permission to access a protected resource. On the other hand, **OpenID Connect** is an authentication protocol and an extension built on top of the OAuth 2 protocol. Using OAuth 2, a client can request access tokens from an STS and then communicate with APIs.

IdentityServer4 can manage or handle authentication and authorization requirements through its framework and implementation of OAuth 2 and OpenID Connect.

That's a quick introduction to IdentityServer4 now; let's proceed to **CIAM**.

Customer identity and access management (CIAM)

Security is complex, and you must get it right. So today, developers trust authentication providers to help secure their applications. **Auth0**, **AAD B2C**, **Okta**, **AWS Incognito**, and **GCP Identity Platform** are popular auth providers, also known as **Identity as a Service (IDaaS)**. So, why should you consider using an auth provider?

Note:

I'm currently an Auth0 ambassador. No, I'm not an employee of the company, nor do I get any monetary compensation. Although occasionally I do get some swag and other cool perks whenever I speak at a conference and mention them. *But* I became an Auth0 ambassador precisely because I've been using it and recommending it.

First, security is complicated, and it's not hard to get wrong. It can feel like you're in a giant puzzle getting all of the options right. By choosing a cloud identity provider that specializes in security, you can trust that the security engineers of an auth provider know what they're doing and you can use your time to focus on developing your application. Similarly, you can customize providers to your needs. Hence, they integrate nicely with your application. So, what about pricing? Well, pricing varies, but some auth providers offer a free tier that works for many types of applications.

I recommend that you use Auth0 because it's easy to use, is the most popular, and has excellent online documentation. It also offers excellent libraries and SDKs for creating single-page apps, and the price is reasonable because it provides a generous free tier. Auth0's free-tier pricing supports up to 7k active users. If you're wondering what features are available in the paid tiers, check out their website, but the free tier is already enough for most applications.

The terminologies *IDaaS* and *cloud identity providers* are being replaced by the acronym CIAM.

OK, so what is **CIAM**, or **customer identity and access management**?

CIAM sits at the intersection of customer experience, security, and analytics. Providing a frictionless way for users to onboard and log in is essential for building customer loyalty and driving conversions. Taking steps to prevent data breaches and protect sensitive data from malicious intrusion is the focus of CIAM for compliance with data privacy laws and security policies. Not to mention that compiling user data into one or a single source of truth is crucial to understanding your customers.

Many companies choose to engage a third-party IDaaS rather than building a solution in-house from scratch.

Companies need **identity and access management (IAM)** solutions for several classes of end users: customers, employees, and enterprise customers. But each type of user requires a different balance of **user experience (UX)** and security. Hence, CIAM solutions provide a superb set of features distinct from workforce identity solutions or B2B.

Here are four features that comprise modern CIAM solutions. No two CIAM solutions offer exactly the same features, but if you're signing up or purchasing a CIAM platform, it should have the following criteria:

- **Scalability:** CIAM has to scale from thousands to millions and even billions of users.
- **Single sign-on:** Allows users to log in or sign in to an app and automatically be signed in to a set of different applications.

- **Multi-factor authentication:** MFA is a more secure means of authenticating a user's identity rather than the common username and password combination.
- **Centralized user management:** Your insights into your end users or customers can be an excellent competitive advantage, but only if the data is accessible, organized, and accurate.

So that's the intro to CIAM. Now, let's move on to the security implementation in our application in the next section.

Authentication implementation using JWT

The **JWT** or **JSON Web Token** is a type of token for carrying identity data between machines. It is supported by different programming languages, an industry standard, and can be easily passed around. A JWT is self-contained, and it holds the needed identity information within itself, as shown in the following figure:

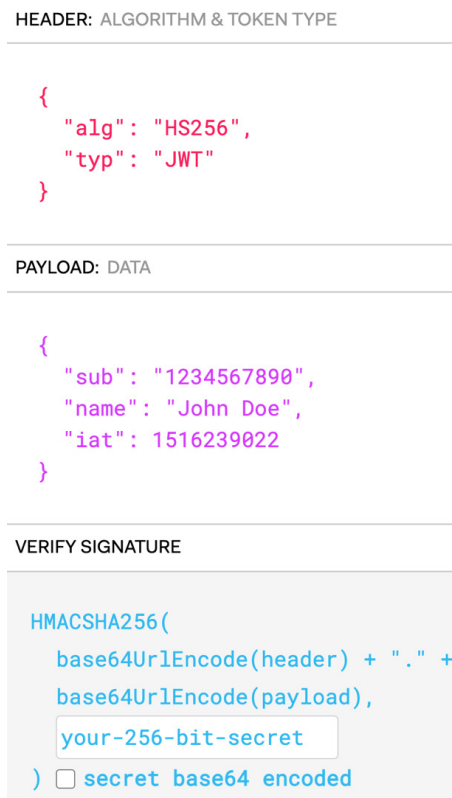


Figure 9.4 – Parts of a JWT

The preceding *Figure 9.4* shows the three parts of the JWT: the header, payload, and signature. The header has two properties. One is `alg`, which is short for algorithm, which determines the algorithm used for encoding this token. `typ` is `JWT`. We don't have to worry about this header because this is just a standard.

What matters to us is the second part, which is the payload. So here, we have a JSON object with three properties: `sub`, usually a user ID, `name`, and `iat`, which is when the token was generated. What you need to know here is that the payload includes public properties about the user, like how on your passport you have some properties about yourself, such as name, date of birth, and birthplace. So, we have the same concept on a JWT. We can include a few basic public properties about the user. Hence, every time we send a token from the client to the server, we can easily extract the user ID from the payload. If you need to know the user's name, you can simply extract that from the payload as well, meaning you don't have to query the database, send the user ID to get a user object, and then extract the name property.

The third part of the JWT is the digital signature. The digital signature is created based on the JWT's header and payload, along with the secret or private key. Like I mentioned earlier, the secret or private key is only available on the server.

Please also refer to the source code of this chapter in GitHub for better readability. If a malicious user gets the JWT and modifies the property, the digital signature will be invalid because the JWT content was modified. Now the hacker would need a new digital signature. Still, the hacker cannot generate a digital signature because they will need the private key available only on the server. So, if they don't have access to the server, they cannot create a valid digital signature. When they send this new tampered JWT to the server, the server will decline it. The server will say "not a valid JWT."

So this is how JWT works.

Implementing token-based authentication

Token-based authentication using JWT is what we are going to implement in our application. This implementation generates a JWT, but what I need you to take from this chapter is knowing what kind of architecture you need and planning what type of security implementation is appropriate in your application in the future. You might need ASP.NET Core Identity, IdentityServer4, or CIAM.

So, let's start adding security to our application:

1. Create a new .NET 5 class library inside the infrastructure directory and name it `Travel.Identity`. After creating the library, delete the default `Class1.cs` file that you will find there.

We also need to create references. Create a reference from this project, `Travel.Identity`, to the `Travel.Application` project and from the `Travel.WebApi` project to this project.

Then install the following two NuGet packages inside the `Travel.Identity` project:

```
Microsoft.AspNetCore.Authentication.JwtBearer
```

This NuGet package is middleware that allows a .NET application to receive an OpenID Connect token. The following NuGet package is middleware that allows a .NET application to support the OpenID authentication workflow:

```
Microsoft.AspNetCore.Authentication.OpenIdConnect
```

The preceding NuGet packages are currently installed in the `Travel.WebApi` project, but we need them in the `Travel.Identity` project and not in the `Travel.WebApi` project. That being said, delete the `JwtBearer` and `OpenIdConnect` packages that are installed in the `Travel.WebApi` project.

2. Next, we go to the `Travel.Domain` project and create a new C# file with the name `User.cs` inside the `Entities` folder:

```
namespace Travel.Domain.Entities
{
    public class User
    {
        public int Id { get; set; }
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public string Username { get; set; }

        public string Password { get; set; }
    }
}
```

The preceding code block is a simple `User` entity with `Id`, `FirstName`, `LastName`, `Username`, and `Password`.

- Now, let's go back to the `Travel.Identity` project and create a folder in its root directory. Name the folder `Helpers`. Create three C# files inside the `Helpers` folder and name them `AuthSettings.cs`, `JwtMiddleware.cs`, and `AuthorizeAttribute.cs`, like so:

```
namespace Travel.Identity.Helpers
{
    public class AuthSettings
    {
        public string Secret { get; set; }
    }
}
```

The preceding block of code is for the secret configuration that we will write later inside the `appsettings.json` file.

The following code block from `JwtMiddleware.cs` is custom middleware that detects and extracts the authorization header from the HTTP request:

```
...
using System.IdentityModel.Tokens.Jwt;
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.Options;
using System.IdentityModel.Tokens;
using Travel.Application.Common.Interfaces;

namespace Travel.Identity.Helpers
{
    public class JwtMiddleware
    {
        private readonly RequestDelegate _next;
        private readonly AuthSettings _authSettings;
        public JwtMiddleware(RequestDelegate next,
            IOptions<AuthSettings> appSettings)
        {
            ...
        }

        public async Task Invoke(HttpContext context,
```

```
        IUserService userService)
    {
        ...
    }

    private void AttachUserToContext(HttpContext
        context, IUserService userService,
        string token)
    {
        ...
    }
    catch
    {
    }
}
}
```

The preceding block of code also validates the token extracted from the header of the sender application, which I truncated and moved here:

```
private void AttachUserToContext(HttpContext context,
    IUserService userService, string token)
{
    try
    {
        var tokenHandler = new
            JwtSecurityTokenHandler();
        byte[] key = Encoding.ASCII.GetBytes
            (_authSettings.Secret);
        tokenHandler.ValidateToken(token, new
            TokenValidationParameters
            {
                ValidateIssuerSigningKey = true,
                IssuerSigningKey = new
                    SymmetricSecurityKey(key),
                ValidateIssuer = false,
                ValidateAudience = false,
```

```

        ClockSkew = TimeSpan.Zero
    }, out var validatedToken);
    var jwtToken =
        (JwtSecurityToken)validatedToken;
    var userId = int.Parse
        (jwtToken.Claims.First(c =>
            c.Type == "id").Value);

    context.Items["User"] =
        userService.GetById(userId);
    } catch { }
}

```

Suppose validation is correct; `userService.GetById` is invoked to get the data of the user.

The following code block from `AuthorizeAttribute.cs` is a custom attribute that lets you annotate a class or a method with `Authorize`:

```

...
namespace Travel.Identity.Helpers
{
    [AttributeUsage(AttributeTargets.Class
        | AttributeTargets.Method)]
    public class AuthorizeAttribute : Attribute,
        IAuthorizationFilter
    {
        public void OnAuthorization
            (AuthorizationFilterContext context)
        {
            var user =
                (User)context.HttpContext.Items["User"];
            if (user == null)
            {
                context.Result = new JsonResult(new {
                    message = "Unauthorized" })
                {
                    StatusCode =
                        StatusCodes.Status401Unauthorized
                };
            }
        }
    }
}

```

```
        }  
    }  
}
```

If the request that hits an annotated route or controller is not authenticated, the controller will send a JSON object with an `Unauthorized` message.

4. Now let's go to the `Travel.Application` project. Find the `Dtos` directory, create a folder inside of that directory, and name it `User`. Then create two C# files inside of the `User` folder and name them `AuthenticateResponse.cs` and `AuthenticateRequest.cs`, like so:

```
namespace Travel.Application.Dtos.User  
{  
    public class AuthenticateResponse  
    {  
        public int Id { get; set; }  
        public string FirstName { get; set; }  
        public string LastName { get; set; }  
        public string Username { get; set; }  
        public string Token { get; set; }  
  
        public AuthenticateResponse  
            (Domain.Entities.User user, string token)  
        {  
            Id = user.Id;  
            FirstName = user.FirstName;  
            LastName = user.LastName;  
            Username = user.Username;  
            Token = token;  
        }  
    }  
}
```

This User model is the shape of the response for an authenticated user after successful login:

```
using System.ComponentModel.DataAnnotations;

namespace Travel.Application.Dtos.User
{
    public class AuthenticateRequest
    {
        [Required]
        public string Username { get; set; }

        [Required]
        public string Password { get; set; }
    }
}
```

The preceding block of code requires the login request to have Username and Password properties or keys in the request body.

5. Now let's create an interface for the user service we will create later. Look for the Common directory inside of the Travel.Application project. Now there's the Interfaces directory inside the Common directory; create an interface inside the Interfaces folder and name it IUserService.cs like so:

```
using Travel.Application.Dtos.User;
using Travel.Domain.Entities;

namespace Travel.Application.Common.Interfaces
{
    public interface IUserService
    {
        AuthenticateResponse Authenticate
            (AuthenticateRequest model);
        User GetById(int id);
    }
}
```

After creating the `IUserService.cs` file, let's create an implementation of that interface. Go back to the `Travel.Identity` project and create a `Services` folder inside the project root directory. Then create a new C# file and name it `UserService.cs`:

```
...
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;
namespace Travel.Identity.Services
{
    public class UserService : IUserService
    {
        private readonly List<User> _users = new
            List<User>
        {
            new User {...}
        };
        ...
        public AuthenticateResponse Authenticate
            (AuthenticateRequest model)
        {
            ...
        }
        public User GetById(int id) =>
            _users.FirstOrDefault(u => u.Id == id);
        private string GenerateJwtToken(User user)
        {
            ...
        }
    }
}
```


The preceding block of code is the implementation of `IUserService.cs` we created earlier. I've truncated the business logic of the `Authenticate` and `GenerateJwtToken` methods for readability purposes. Here is the complete code of the `Authenticate` method as well as the `GenerateJwtToken` method:

```
public AuthenticateResponse Authenticate
    (AuthenticateRequest model)
{
    var user = _users.SingleOrDefault(u =>
        u.Username == model.Username
        && u.Password == model.Password);

    if (user == null)
        return null;

    var token = GenerateJwtToken(user);

    return new AuthenticateResponse(user,
        token);
}
```

This implementation authenticates User by checking whether the user exists in the database. However, we hardcoded the user object here to simplify the implementation:

```
private string GenerateJwtToken(User user)
{
    byte[] key = Encoding.ASCII.GetBytes
        (_authSettings.Secret);
    var tokenDescriptor = new
        SecurityTokenDescriptor
    {
        Subject = new ClaimsIdentity(new[] {
            new Claim("id",
                user.Id.ToString()) },
        Expires = DateTime.UtcNow.AddDays(1),
        SigningCredentials = new
            SigningCredentials(new
                SymmetricSecurityKey(key),
```

```
        SecurityAlgorithms.HmacSha256Signature
    )
};

var tokenHandler = new
    JwtSecurityTokenHandler();
var token = tokenHandler.CreateToken
    (tokenDescriptor);

return tokenHandler.WriteToken(token);
}
```

The `GenerateJwtToken` method generates a JWT by using the value of `Secret` in the `appsettings.json` file. It also uses `JwtSecurityTokenHandler` and `SecurityTokenDescriptor`, which create the claims or payload of the token. You will notice here that `SecurityTokenDescriptor` invokes `SigningCredentials` with `key` and `algorithm` arguments to sign the token.

6. Now let's create a `DependencyInjection` class that uses a static `IServiceCollection` for the `Travel.Identity` project. Create a C# class in the root directory of the `Travel.Identity` project like so:

```
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Travel.Application.Common.Interfaces;
using Travel.Identity.Helpers;
using Travel.Identity.Services;

namespace Travel.Identity
{
    public static class DependencyInjection
    {
        public static IServiceCollection
            AddInfrastructureIdentity
            (this IServiceCollection services,
             IConfiguration config)
        {
            services.Configure<AuthSettings>
```

```

        (config.GetSection(nameof(AuthSettings))) ;
        services.AddScoped<IUserService, UserService>();

        return services;
    }
}
}

```

Here is the registered service in our **dependency injection (DI)** container that we will register in `Startup.cs` in a bit. We are getting the object of `AuthSettings` in the `appsettings.json` file and registering a scoped lifetime for `UserService`. The `Secret` key of `AuthSettings` is a sensitive string that must be stored in an environment variable or Azure Key Vault to protect it.

7. Next, we create a C# file and name it `UserController` inside the `Travel.WebApi` project, specifically inside the `v1` folder of the `Controllers` directory, like so:

```

using Microsoft.AspNetCore.Mvc;
using Travel.Application.Common.Interfaces;
using Travel.Application.Dtos.User;
namespace Travel.WebApi.Controllers.v1
{
    [ApiVersion("1.0")]
    [ApiController]
    [Route("api/v{version:apiVersion}/{controller}")]
    public class UsersController : ControllerBase
    {
        private readonly IUserService _userService;
        public UsersController(IUserService
            userService) => _userService = userService;
        [HttpPost("auth")]
        public IActionResult Authenticate([FromBody]
            AuthenticateRequest model)
        {
            var response =
                _userService.Authenticate(model);
            if (response == null)

```

```
        return BadRequest(new { message =  
            "Username or password is incorrect"  
        });  
        return Ok(response);  
    }  
}
```

This controller does not require authorization since it is intended for logging in. It also uses the v1 API to make it consistent. The controller authenticates the user and responds with the JWT for User.

8. Now, let's protect the rest of the controllers by adding the custom attribute in `ApiController`. Let's update `ApiController`'s code:

```
namespace Travel.WebApi.Controllers.v1  
{  
    [Authorize]  
    [ApiVersion("1.0")]  
    [ApiController]  
    [Route("api/v{version:apiVersion}/{controller}")]  
    public abstract class ApiController :  
        ControllerBase  
    {  
        private IMediator _mediator;  
        protected IMediator Mediator => _mediator ??=  
            HttpContext.RequestServices.GetService  
                <IMediator>();  
    }  
}
```

We are annotating `ApiController` with the `Authorize` attribute, the custom attribute that we created earlier. The attribute protects all the children classes of `ApiController` from authenticated consumers or users.

9. Now the last file to update is `Startup.cs` with the following code:

```
public void ConfigureServices(IServiceCollection
services)
{
    services.AddApplication();
    services.AddInfrastructureData();
    services.AddInfrastructureShared
(Configuration);
    services.AddInfrastructureIdentity
(Configuration);

    services.AddHttpContextAccessor();
    services.AddControllers();

    ...
}
```

Add the `services.AddInfrastructureIdentity(Configuration);` method below the `services.`

`AddInfrastructureShared(Configuration);` method.

10. Let's also update the SwaggerGen configuration. The code that we are adding will allow us to enter a JWT in the Swagger HTTP requests header through a dialog box:

```
services.AddSwaggerGen(c =>
{
    c.OperationFilter
<SwaggerDefaultValues>();
    c.AddSecurityDefinition("Bearer", new
OpenApiSecurityScheme
{
        Description = "JWT Authorization header
using the Bearer scheme.",
        Name = "Authorization",
        In = ParameterLocation.Header,
        Type = SecuritySchemeType.Http,
        Scheme = "bearer"
    });
});
```

```
c.AddSecurityRequirement(new
    OpenApiSecurityRequirement
    {
        {
            new OpenApiSecurityScheme
            {
                Reference = new OpenApiReference
                {
                    Type =
                        ReferenceType.SecurityScheme,
                    Id = "Bearer"
                }
            }, new List<string>()
        }
    }
    ));
```

AddSecurityDefinition describes how the API is protected, while AddSecurityRequirement adds a global security requirement in the APIs.

Also, update the middleware like so:

```
public void Configure(IApplicationBuilder app,
    IWebHostEnvironment env, IApiVersionDescriptionProvider
    provider)
{
    ...
    app.UseRouting();

    app.UseMiddleware<JwtMiddleware>();

    app.UseAuthorization();

    ...
}
```

Insert the custom `app.UseMiddleware<JwtMiddleware>()` middleware, between `app.UseRouting()` and `app.UseAuthorization()`. This code block is the custom middleware that we created earlier.

Now, the `Travel.Identity` project is integrated into the `Travel.WebApi` project.

11. Run the application to see whether the security is working correctly in the application and Swagger.

After running the application, let's check out Swagger UI.

Checking Swagger UI

After setting up the JWT authentication and adding it to Swagger, let's try and see how we would use and test our protected routes in our Swagger UI:

1. Swagger UI has the **Authorize** button for adding credentials or JWT, as shown in the following figure, *Figure 9.5*:

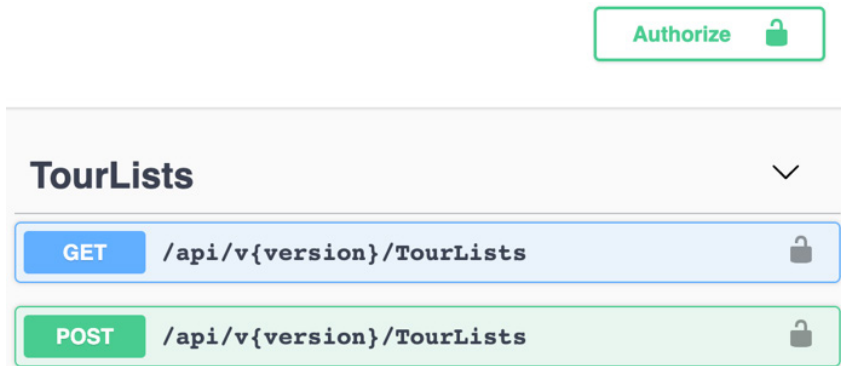


Figure 9.5 – OpenAPI with the security definition and requirement

2. Now let's check out the `/api/v1/Users/auth` route of the `Users` controller:

The image shows the Swagger UI for a POST endpoint. The title is 'Users'. The endpoint is `/api/v{version}/Users/auth`. There is a 'Parameters' section with a table:

Name	Description
version * required string (path)	The requested API version

The value for 'version' is '1.0'. There is a 'Request body' section with a dropdown set to 'application/json'. The request body is a JSON object:

```
{  "username": "yoursuperhero",  "password": "Pass123!"}
```

Figure 9.6 – Signing in using Swagger UI

We can test the `auth` API by sending a POST request with username as `yoursuperhero` and password as `Pass123!`, as shown in the preceding figure.

- Now let's check out the response of the API after sending the POST request:

Responses

Curl

```
curl -X POST "https://localhost:5001/api/v1.0/Users/auth" -H "accept: */*" -H "Content-Type: application/json" -d '{"username\":\"yoursuperhero\",\"password\":\"Pass123!\"}'
```

Request URL

```
https://localhost:5001/api/v1.0/Users/auth
```

Server response

Code	Details
200	Response body <pre>{ "id": 1, "firstName": "Yourname", "lastName": "Yoursurname", "username": "yoursuperhero", "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjEiLCJmYmYiOiJlZ2MDYyNTQzMjMsImV4cCI6MTYwNjM0MDcyMywiaWF0IjoxNjA2MjU0MzIzfQ.UrNvPnibhTob6zPwL_jl0LzTpcGQbCsr" }</pre>

Figure 9.7 – JWT response after successful login

The preceding screenshot shows the JWT from the auth API's response body, which means we are authenticated after sending the username and password. Let's copy the token's value because we will use this in the **Authorize** box of Swagger UI.

4. Go to the `https://jwt.io` website and paste the JWT in the left box, as shown in the following screenshot:



Figure 9.8 – Encoded and decoded JWT

We see the encoded JWT that we receive from the auth API of the application. At the same time, the right side is the decoded JWT. The decoded token shows the header and the payload. The header shows the signature or the encryption algorithm and the type of token. Subsequently, the decoded token also shows the token's payload, which has the user's ID, *not valid before* time, *expiration* time, and *issued at* time.

5. Now let's check out the **Authorize** button of Swagger UI by clicking it:

Available authorizations

✕

Bearer (http, Bearer)

JWT Authorization header using the Bearer scheme.

Value:

eyJhbGciOiJIUzI1NiIsInR5cC

Authorize

Close

Figure 9.9 – Including the JWT in every API testing

Paste the token value in the dialog box to allow Swagger UI to include an **Authorization** header in every request, as shown in the preceding screenshot. You can try to send a GET request to `/api/v1.0/TourLists` using Swagger, or you can use the JWT in Postman.

6. Open up Postman and choose the bearer token under the **Auth** tab. Then paste the token in the input box. Send a GET request to the `TourLists` controller, `https://localhost:5001/api/v1.0/TourLists`:

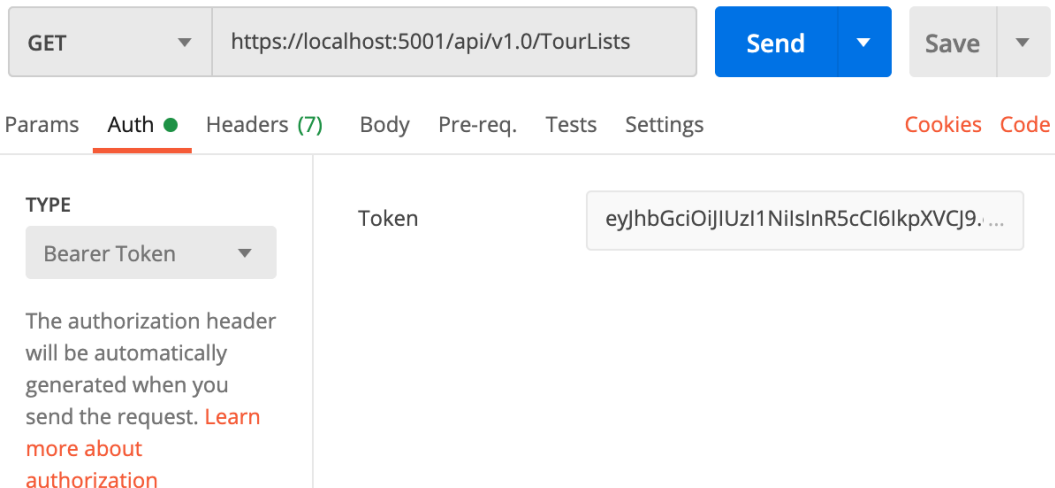


Figure 9.10 – Authorized API testing using Postman

The following screenshot shows a 200 status code with the data that we are expecting:

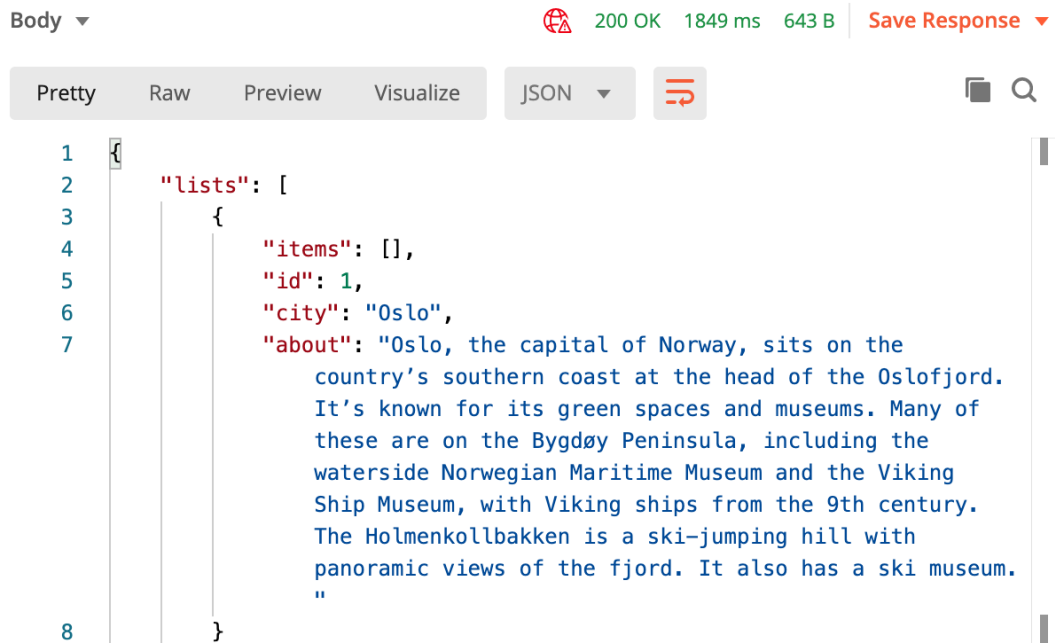


Figure 9.11 – 200 OK response from a protected API

How about if we send the wrong JWT? What do you think the application will send back? Delete some letters in the token and replace them with your name. Then hit the **Send** button to send the request:

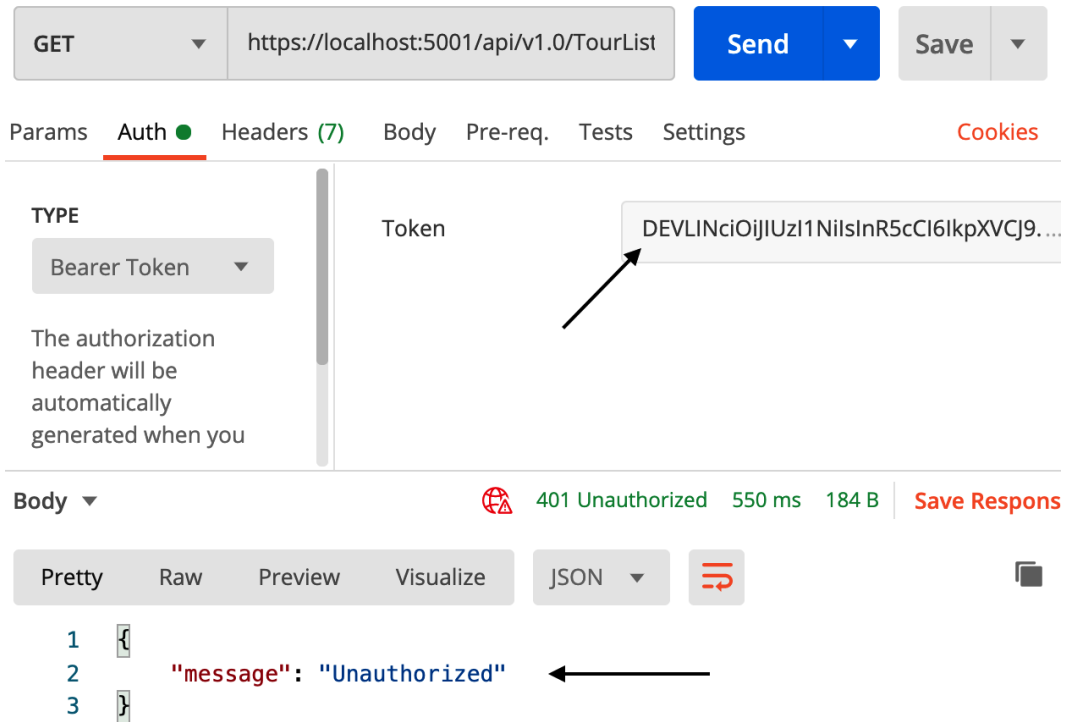


Figure 9.12 – Unauthorized request

You will see that the application responds back with a 401 status code and an **Unauthorized** message, as shown in the preceding screenshot.

That's how you would use Swagger UI with protected routes or endpoints. Now let's finish this chapter by summarizing what we have learned here.

Summary

Here is a summary of this chapter. You've learned that ASP.NET Core Identity is an open source identity framework in .NET that provides you with the ability to manage user authentications and accounts. It has built-in middleware that enables the application to use cookie-based authentication.

You've learned that the IdentityServer4 framework takes away the heavy lifting for implementing OAuth 2 and OpenID Connect in your ASP.NET Core application. A fair use case for IdentityServer4 is when you want a centralized authentication server that authenticates requests from different services using token-based authentication.

You've also learned that CIAM is a cloud-based identity provider that provides companies with analytics, security, and a good customer experience. It's scalable, has centralized user management, and is easy to set up single sign-on and MFA.

You have learned how to secure an ASP.NET Core 5 application using the JWT. The application responds with a token, which has a payload of user information, to an authenticated request.

Lastly, you've also learned how to show protected APIs in Swagger UI and how to include an authorization header in Swagger's every request.

In the next chapter, we will talk about performance improvement using Redis caching.